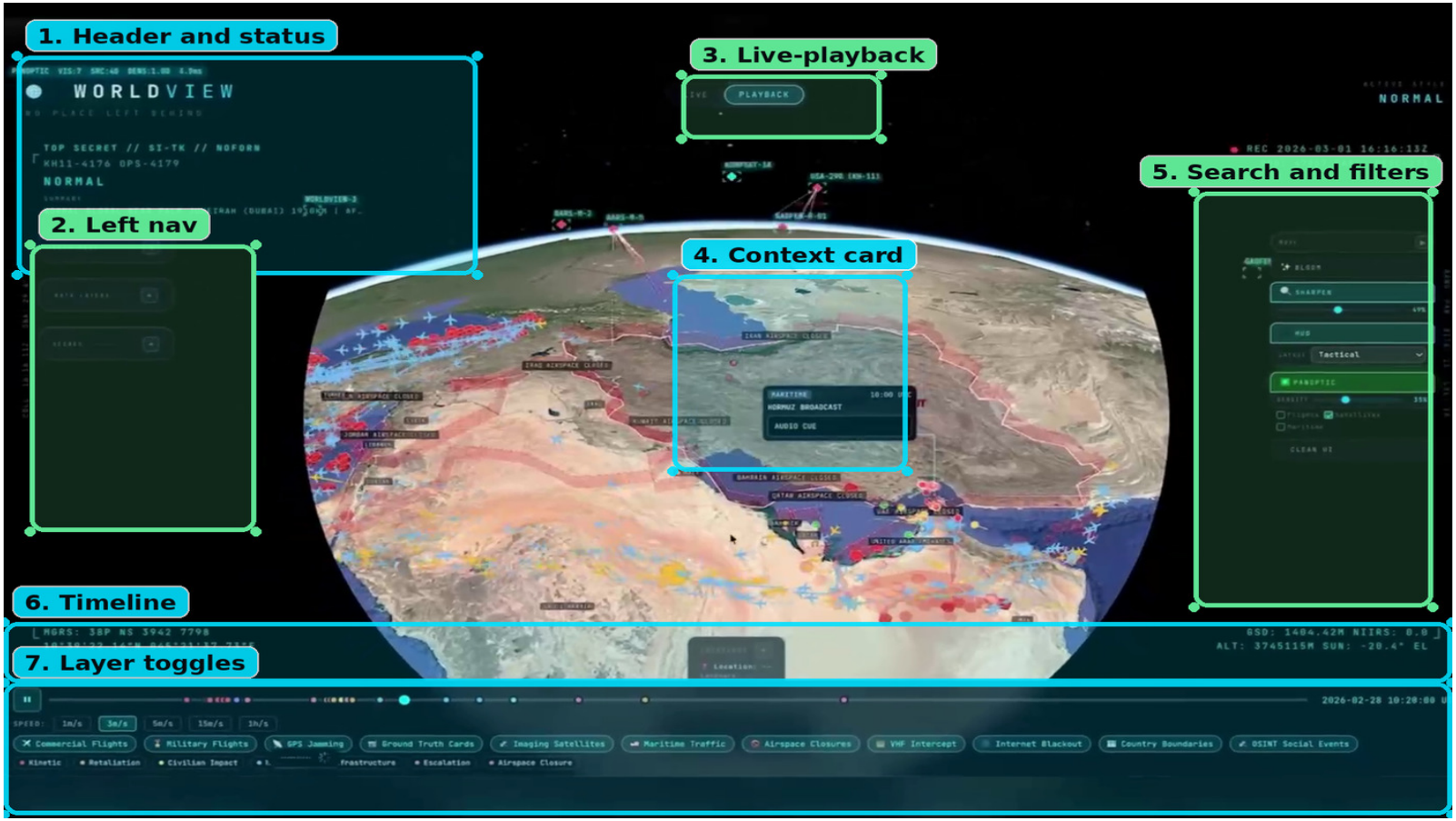
# WORLDVIEW UI teardown

## Annotated reverse-engineering notes from 10 extracted frames

Inference only. Based on visible UI behavior in the screen recording, not source code, network traffic, or live inspection.




### Key takeaways

- Single-page geospatial console with a fixed shell around a 3D globe/terrain viewport.
- Separate control zones for navigation, filters, and playback.
- Render presets appear layered over one shared data model.

# Structural read

## Probable component inventory

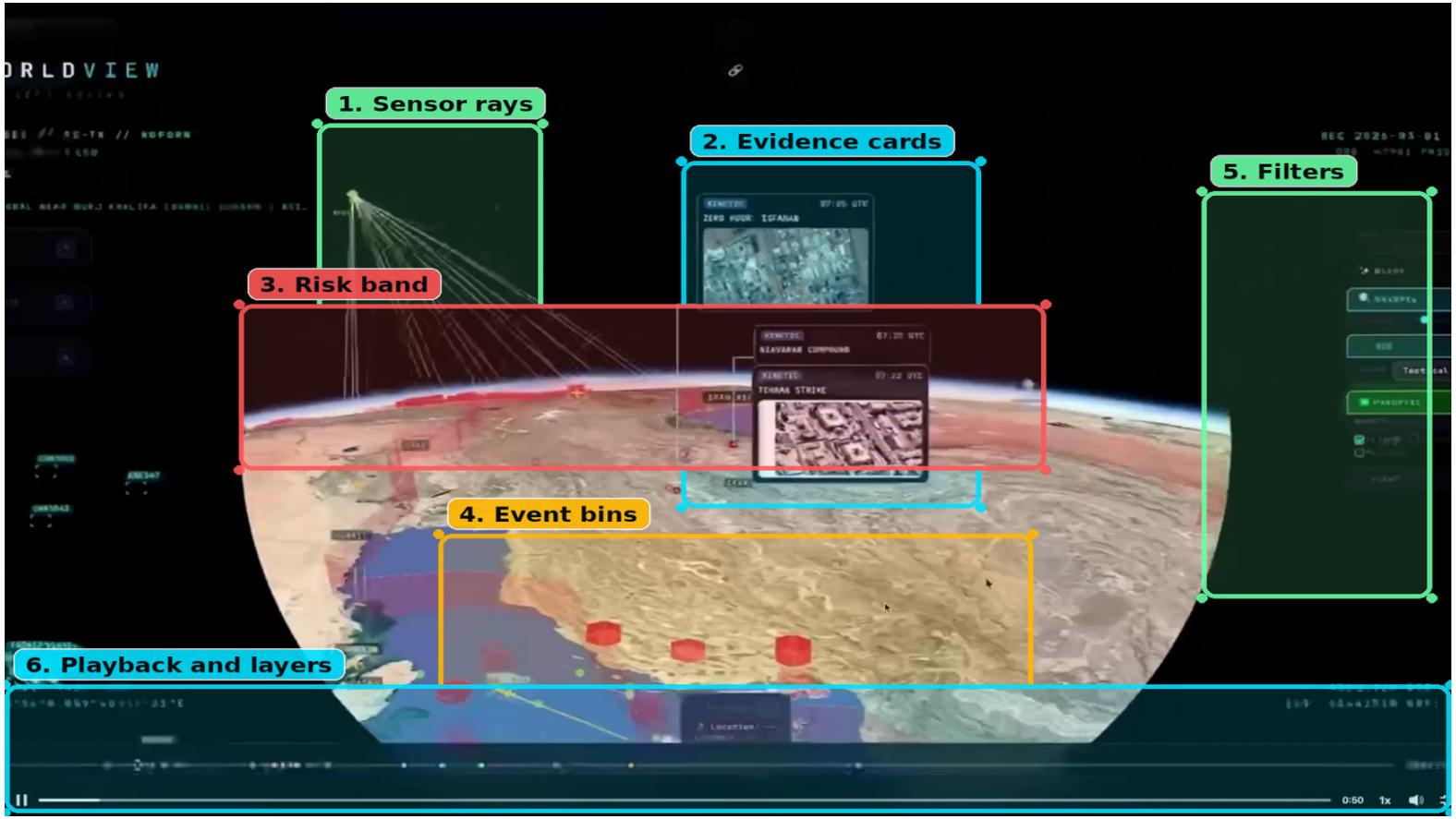
| Component | Likely role | Priority |
|---|---|---|
| Header/status | Scenario metadata, state badges, title, environment | High |
| Left rail | Section drill-downs or task shortcuts | Medium |
| Live/Playback switch | Global mode selection | High |
| Right filter rail | Search, preset, opacity, toggles, reset | High |
| Map canvas | 3D globe/terrain plus overlays | Highest |
| Evidence cards | Typed detail panels anchored to map features | High |
| Timeline rail | Time scrubber, markers, speed presets | High |
| Layer chips | Visibility control and data taxonomy surface | High |

## Likely layer stack

| Layer | Visible cues |
|---|---|
| Base geography | Terrain imagery, coastlines, horizon, local relief |
| Boundaries/labels | Country names, inline region labels, coast/port context |
| Thematic polygons | Airspace closures, blackout zones, shaded risk regions |
| Routes/tracks | Flight splines, maritime paths, movement history |
| Assets/entities | Aircraft glyphs, vessel points, named labels |
| Events/aggregates | Colored points, clusters, red hex bins |
| Off-surface sources | Sensor beams, orbital or tasking nodes |
| Cards/narrative | Thumbnail panels, incident captions, blackout labels |

# 01 - Globe overview with popups

Timestamp: 5.5s




## What this frame reveals

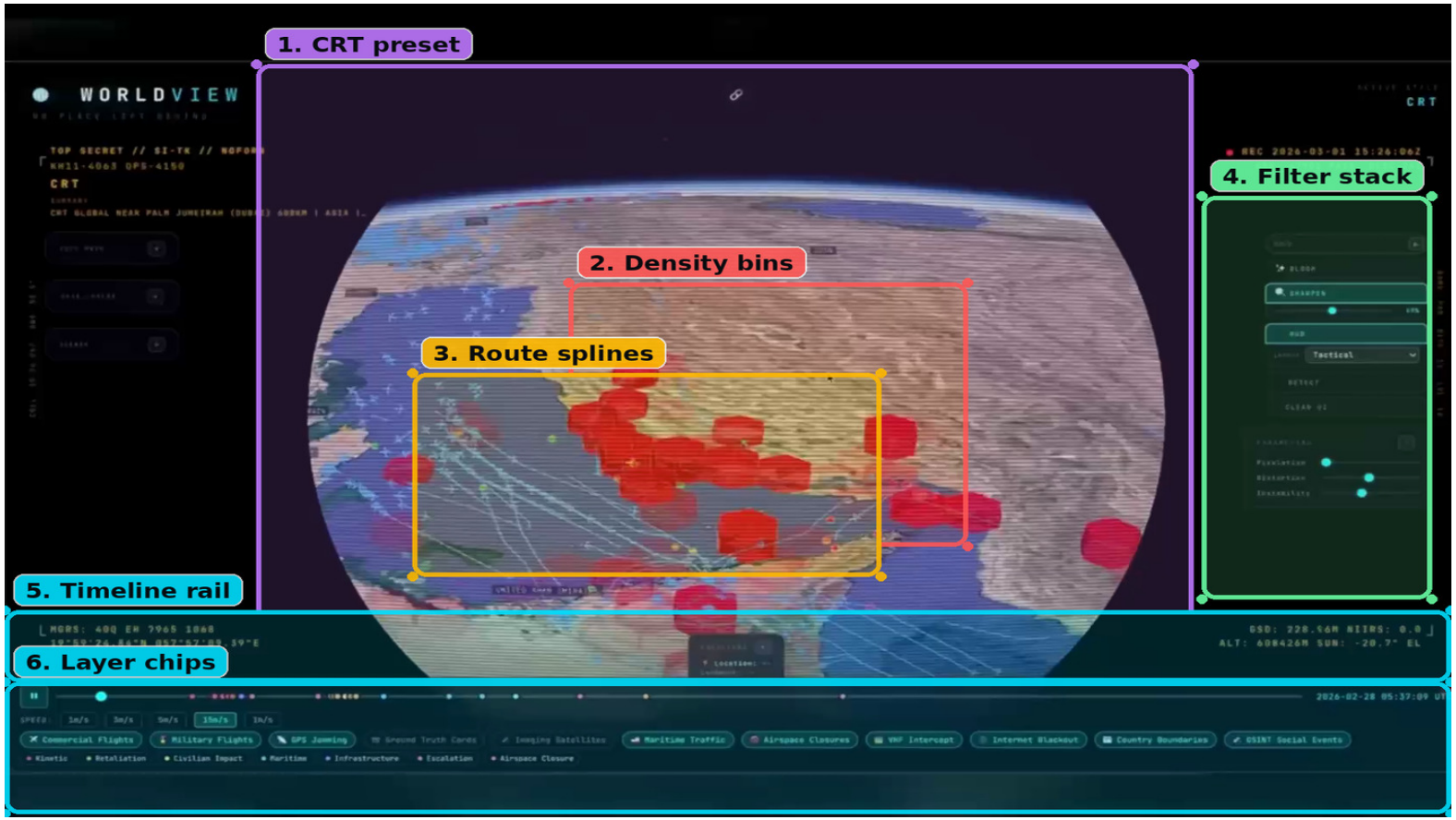
Opening state combines a theater-wide globe, anchored intel cards, and at least one orbital or sensor source. The interface is narrative-first: the user starts with context, evidence, and hazards visible together.

## Top takeaways

- Popup cards are map-anchored but screen-stable enough to remain legible during camera pitch.
- The camera is pitched and horizon-aware, which implies a true 3D globe or 3D terrain scene rather than a flat map only.

# 02 - Operational map overview

Timestamp: 7.8s




## What this frame reveals

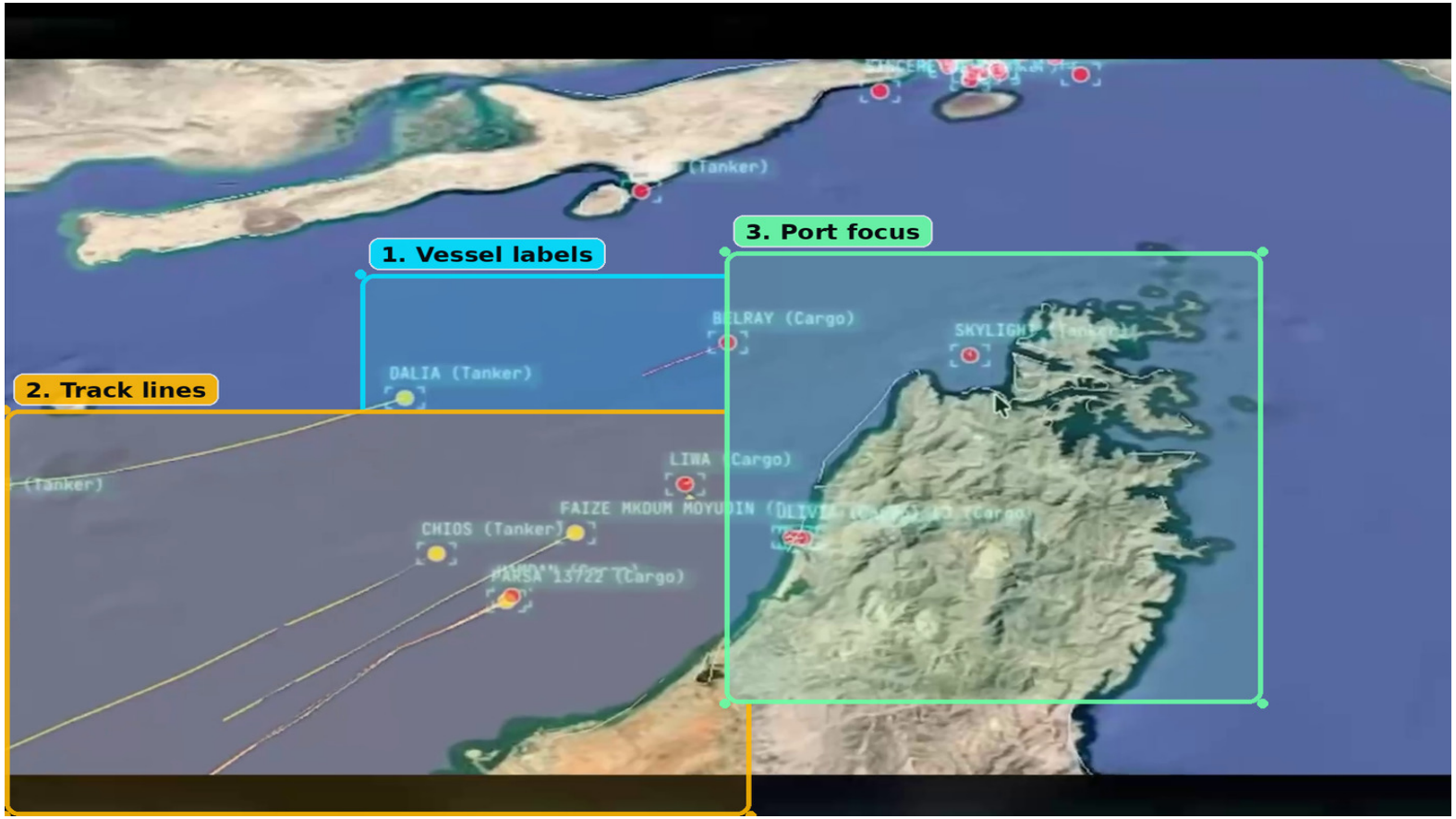
This frame exposes the same shell with a different visual preset. Scanlines and muted contrast suggest a display-mode abstraction, not a one-off effect.

## Top takeaways

- Render mode is likely a top-level preset that changes post-processing while leaving the data layers intact.
- Large red bins appear to encode density or severity; they look generated, not hand-placed.

# 03 - Regional maritime close-up

Timestamp: 12.0s




## What this frame reveals

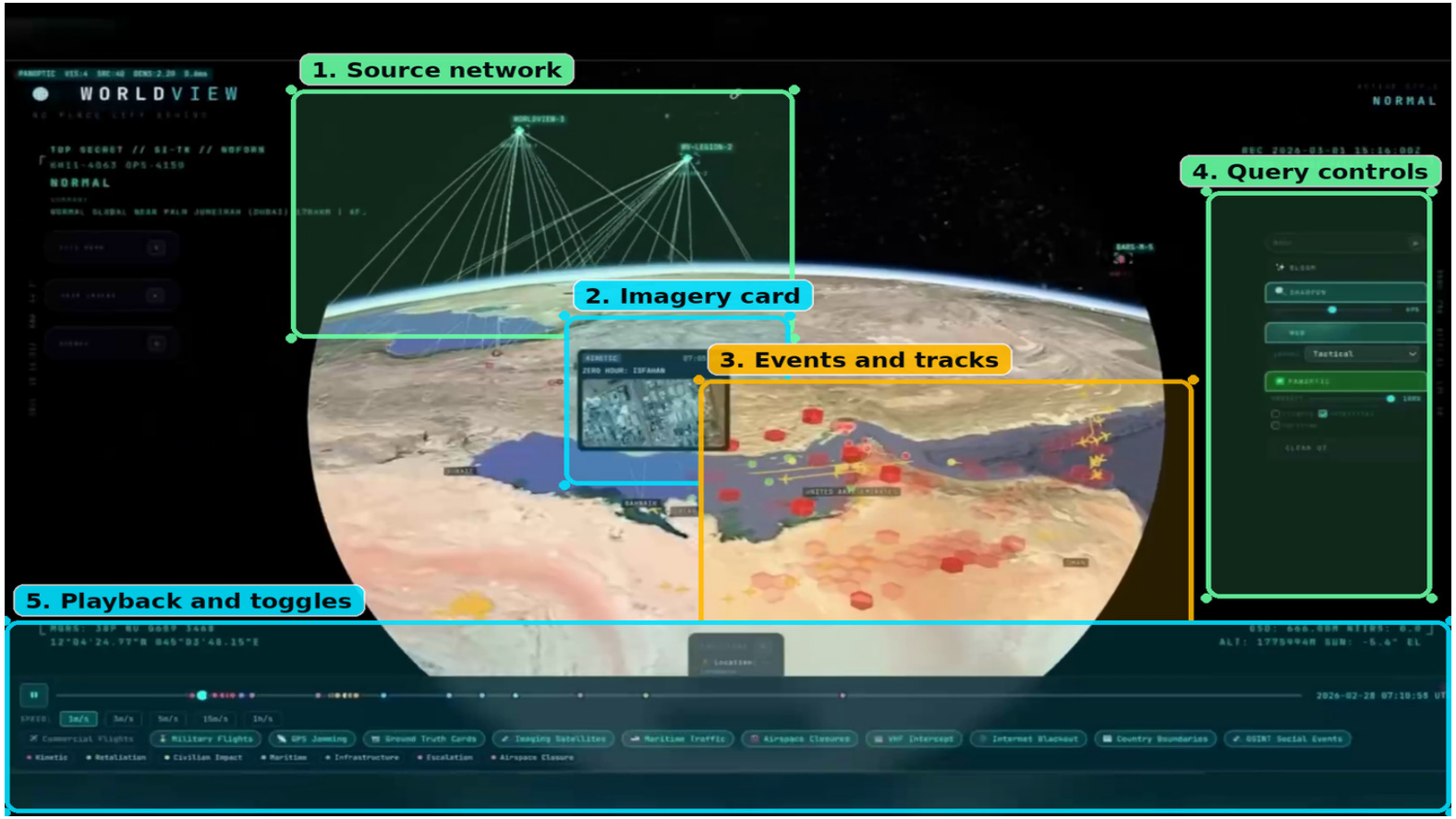
The camera can zoom from macro theater view to a low-altitude local view without leaving the same visual language. Entity labels switch from aggregates to named assets.

## Top takeaways

- Maritime entities and track lines are styled differently from the broad theater overlays, which suggests a typed layer system.
- The coastline and terrain keep depth cues at close range, pointing to terrain tiles rather than a flat raster backdrop.

# 04 - Globe with beam network

Timestamp: 14.0s




## What this frame reveals

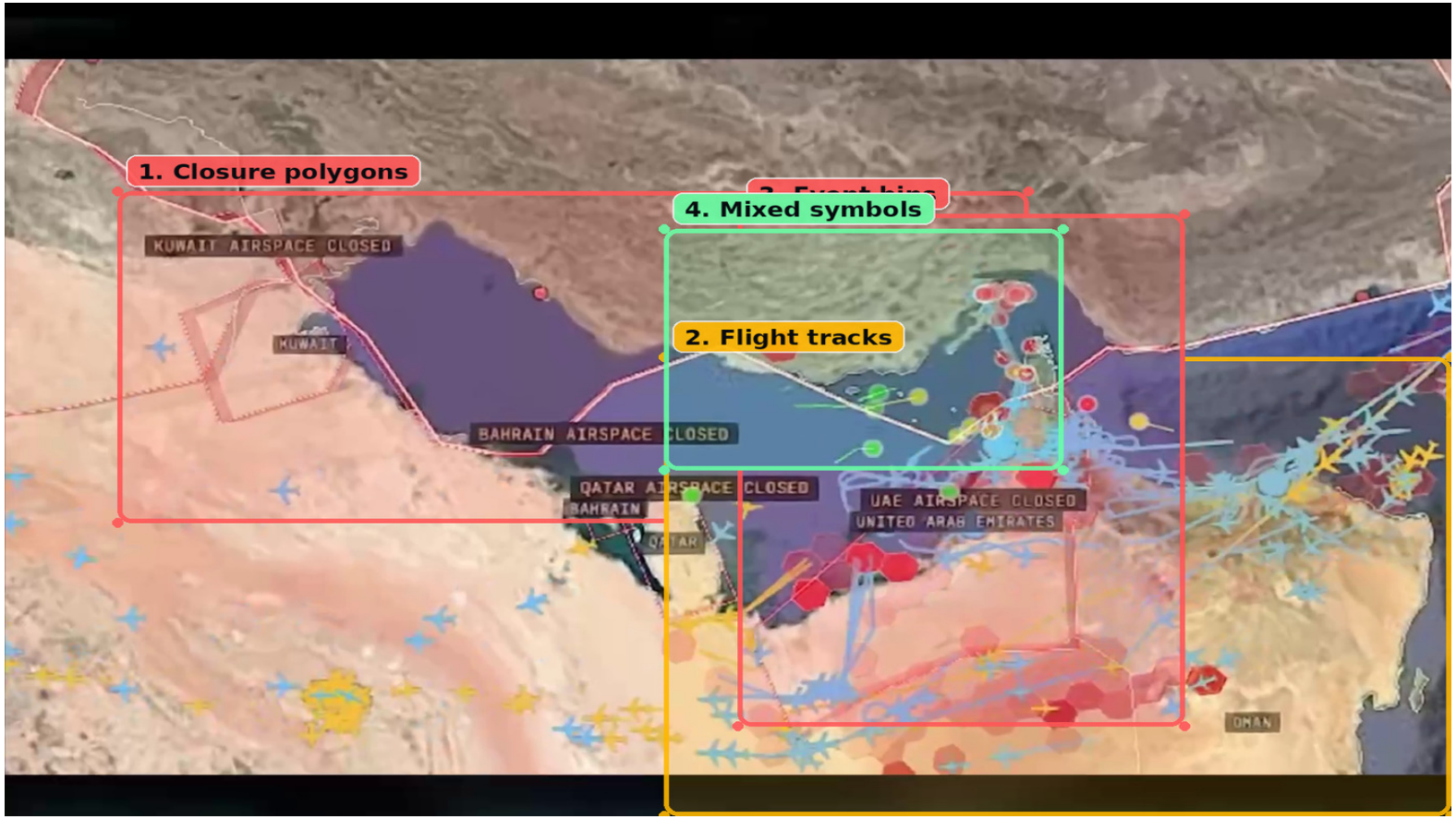
Compared with frame 01, the source layer becomes a network of multiple origins. That suggests the app can show sensor coverage, satellite tasking, or communications links as first-class objects.

## Top takeaways

- Leader lines terminate at labeled sources above the globe, so the scene graph likely contains off-surface objects as well as ground overlays.
- Evidence cards can coexist with macro overlays without collapsing the camera back to 2D.

# 05 - Dense tactical overlay

Timestamp: 18.0s




## What this frame reveals

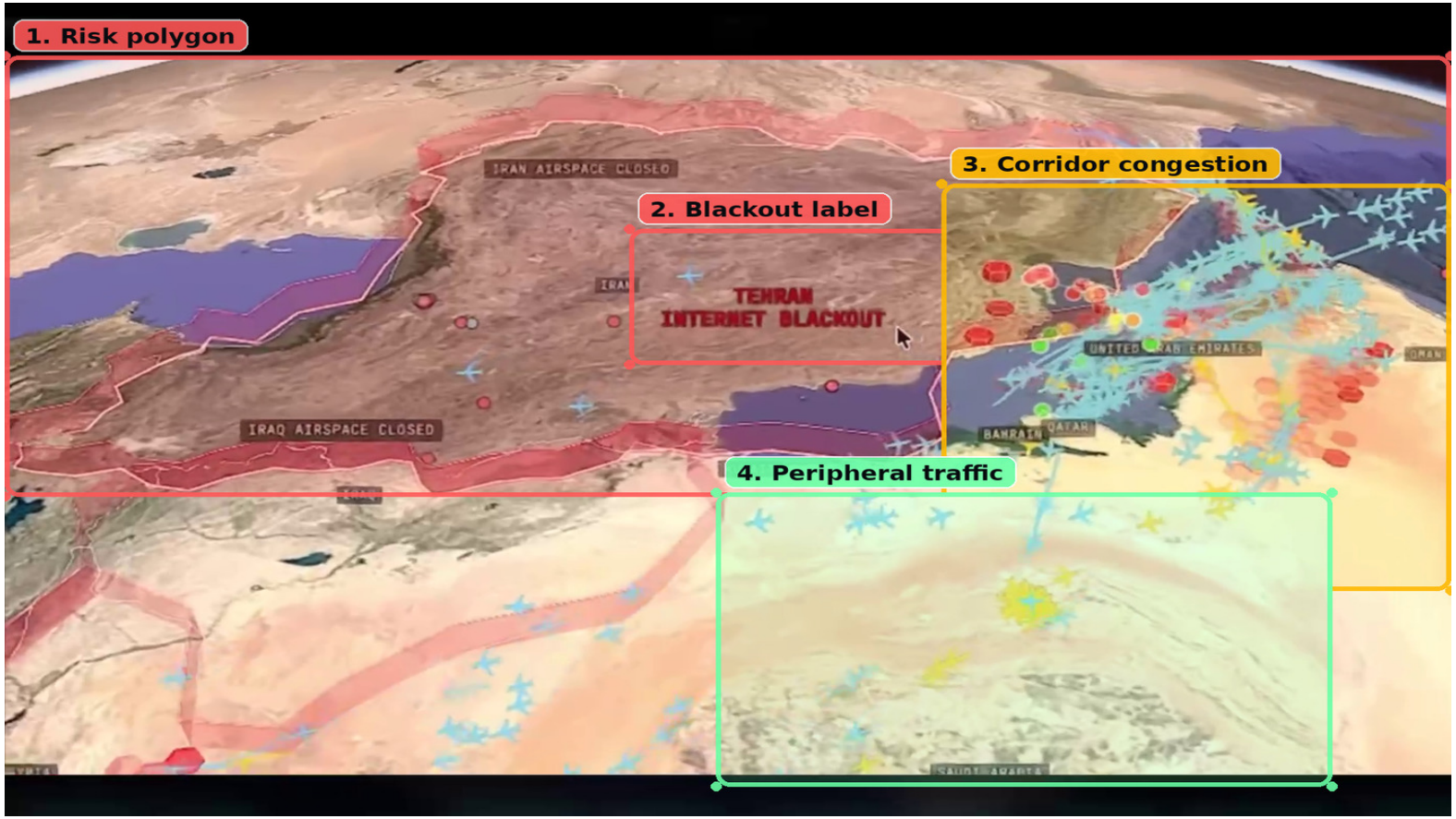
This is the clearest view of the dense overlay state: named closure regions, moving aircraft glyphs, colored event points, and spatial bins all coexist in one tactical frame.

## Top takeaways

- Area alerts are carried by both polygons and inline text banners, which is useful when geometry alone becomes ambiguous.
- Colored points plus hex bins indicate at least two event granularity levels: individual observations and aggregated density.

# 06 - Theater risk overlay

Timestamp: 26.0s




## What this frame reveals

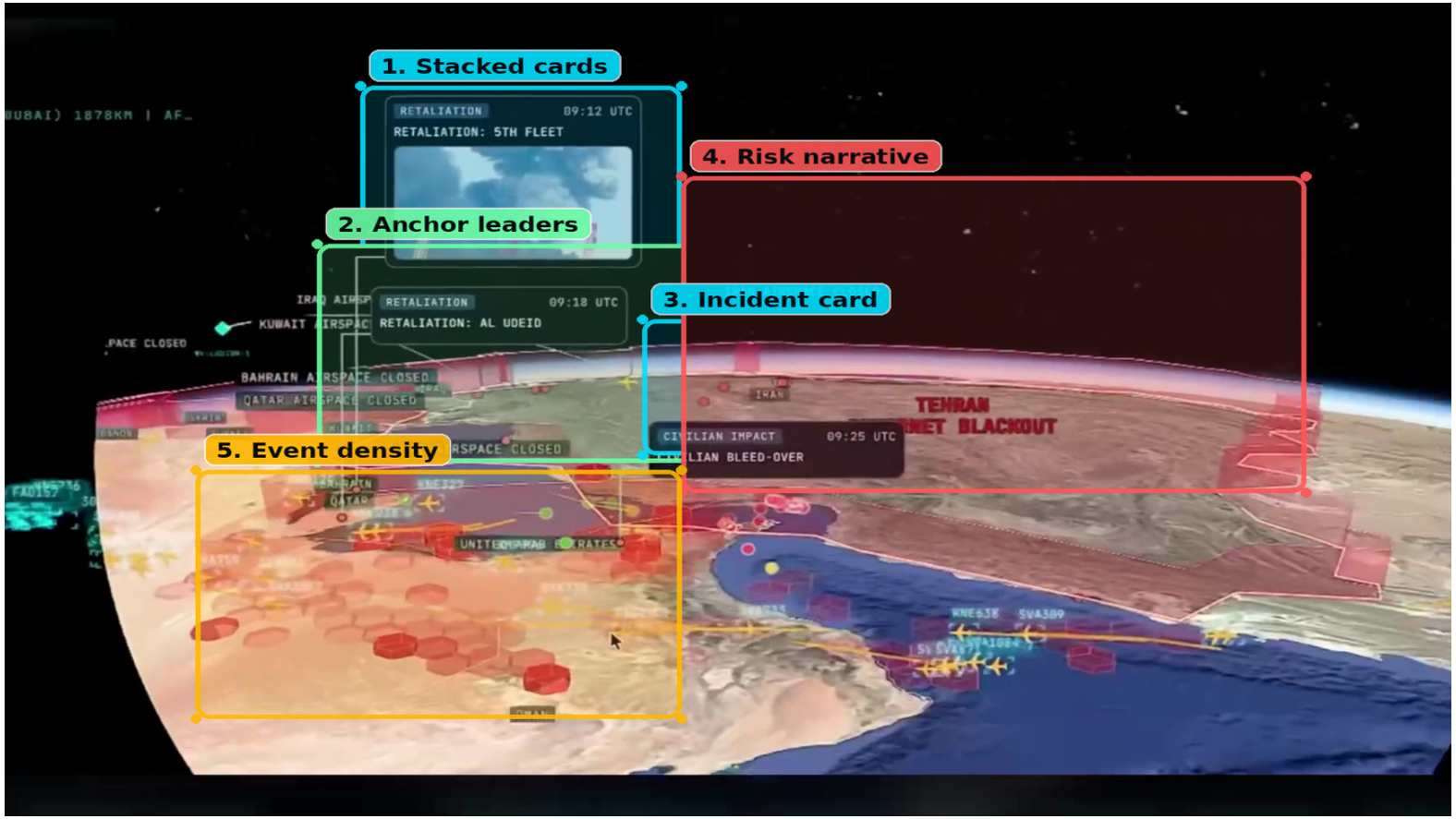
The interface can flatten many local signals into a single theater-level narrative zone. Here, a broad shaded polygon and large label turn scattered incidents into an overall condition.

## Top takeaways

- The app supports thematic overlays that summarize many underlying events as one story-level state.
- Big text directly on the map is likely generated from data categories, not manually composited captions.

# 07 - Multi-card intel overlay

Timestamp: 32.0s




## What this frame reveals

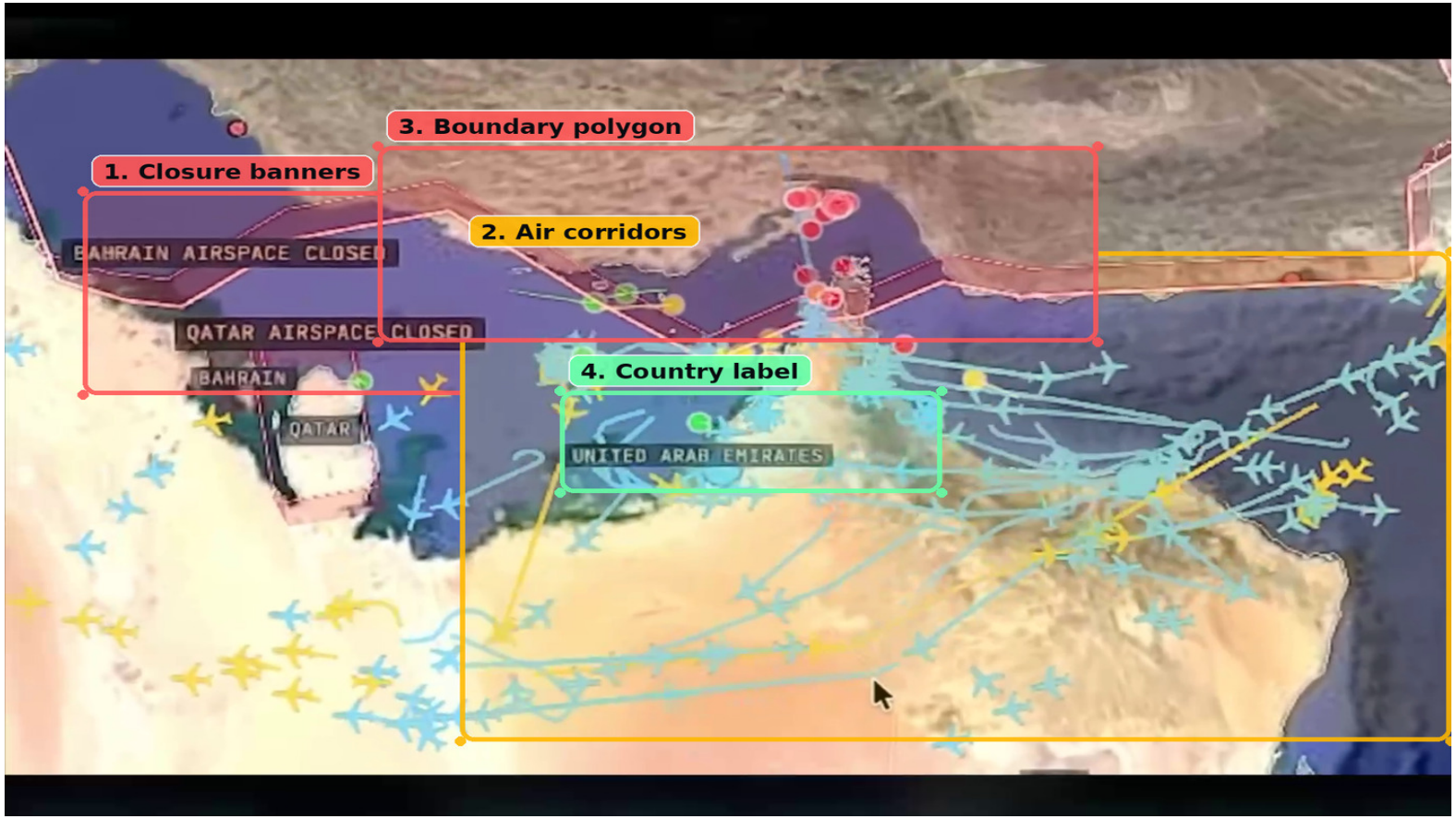
The card system is not limited to one selected object. Multiple evidence cards can stack, connect back to the map, and coexist with broader incident labels.

## Top takeaways

- Cards probably share a reusable component schema: type tag, time, title, media block, and map anchor.
- Leader lines reduce ambiguity when cards overlap dense geometry, so selection state is likely graph-aware.

# 08 - Airspace routes close-up

Timestamp: 38.0s




## What this frame reveals

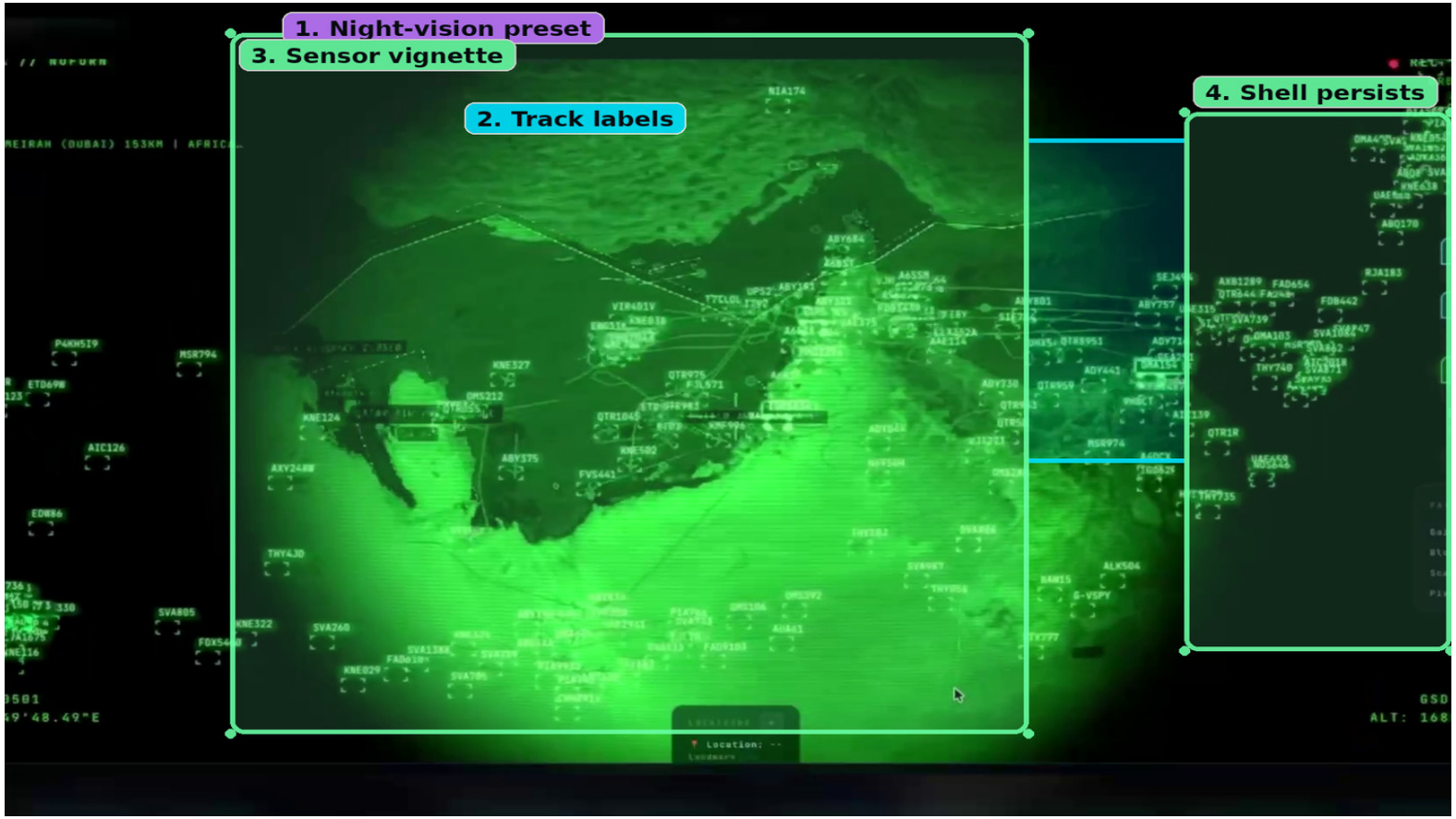
A close-up of the aviation view shows how the app explains route disruption. Text banners, boundary lines, and flight paths all tell the same story at different scales.

## Top takeaways

- Aircraft glyphs inherit line colors, so color likely maps to route class, status, or affiliation.
- Boundary closures use both edge stroke and fill, which makes them readable over water and land.

# 09 - Sensor or night mode

Timestamp: 44.0s




## What this frame reveals

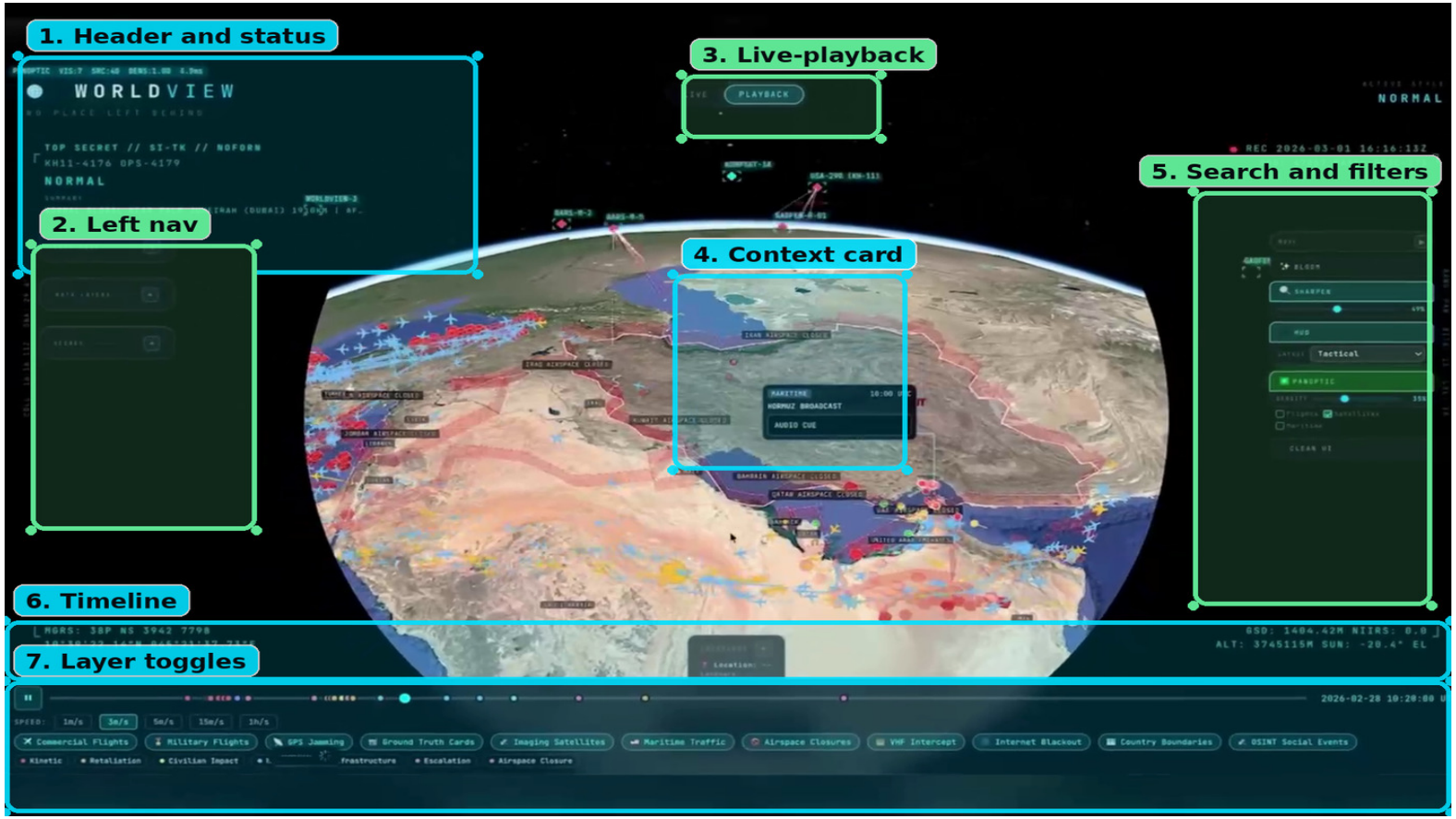
This is almost certainly a render preset layered over the same operational data. The vignette, monochrome palette, and dense labels mimic a sensor feed or night-vision monitor.

## Top takeaways

- Because the shell still peeks through at the edges, the mode is probably a shader or post-processing branch, not a separate application view.
- Dense alphanumeric tags imply the system can switch from thematic storytelling to raw-ish tracking output.

# 10 - Full UI shell overview

Timestamp: 54.0s




## What this frame reveals

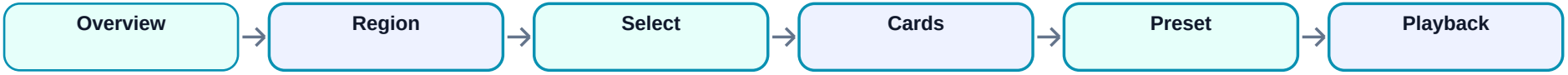
This is the best single frame for reconstructing layout. The interface is a fixed shell wrapped around one main geospatial canvas, with timeline and layers anchored along the bottom.

## Top takeaways

- The app reads like a single-page operational console: fixed chrome, stateful map canvas, and popovers instead of route changes.
- Search, mode, and layer controls live in separate regions, which hints at separate stores or controllers in the frontend.

# Likely interaction and implementation model

## State flow




Stable shell: header, left nav, right filter rail, and bottom playback remain fixed.

Presets likely swap post-processing, while tracks, cards, and polygons share one scene graph.

## Implementation hypotheses

- Map engine is likely Cesium-style globe/terrain or a custom WebGL scene graph.
- Layers look typed and centrally managed for visibility, opacity, legend text, and draw order.
- Playback likely rehydrates tracks, markers, cards, and aggregates from a time-indexed event stream.
- Normal, CRT, and night-vision modes read like post-processing presets over one shared model.

## Suggested rebuild order

- Rebuild screenshot 10 first: shell, transport rail, and layer chips.
- Then reproduce 3D camera and terrain pitch before fine styling.
- Add overlays in this order: closures, tracks, entities, bins, cards, then beams.